

Programmation de systèmes mobiles — ANDROID — *CAI*

Cédric Buche

ENIB

30 mars 2020

- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement
- 4 Interface Graphique
- 5 Manipulez les Périphériques & Capteurs
- 6 Labo
- 7 Communiquez en Bluetooth & SMS
- 8 Publiez & Monétisez

- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement
- 4 Interface Graphique
- 5 Manipulez les Périphériques & Capteurs
- 6 Labo
- 7 Communiquez en Bluetooth & SMS
- 8 Publiez & Monétisez

Mobile Devices



FIGURE – Mobile Devices

Mobile App

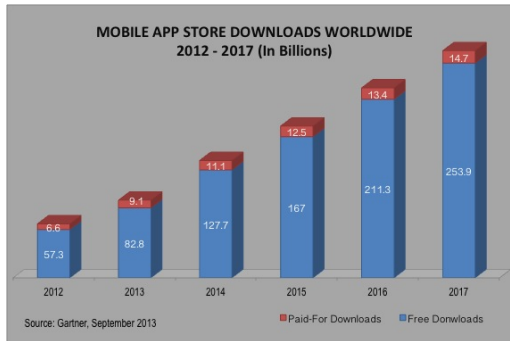


FIGURE – Mobile app, worldwide, 2012-2017

Mobile App vs Web App

	Application mobile (native)	Application web
Portabilité	Développement spécifique à chaque plateforme	Navigateur Web
Développement/coût	Nécessite un SDK + connaissance d'un langage spécifique	Langage du Web (HTML / JS / CSS /PHP...)
Mises à jour	Soumission à un magasin d'applications et éventuelle validation + retéléchargement par le client	Mise à jour rapide en mettant à jour tout simplement les fichiers sur le serveur Web
Disponibilité	Mode online et offline	Nécessite obligatoirement une connexion internet
Fonctionnalités	Utilise toutes les fonctionnalités du mobile (GPS, voix, notifications, contacts...)	Limitées aux possibilités du navigateur

OS market share

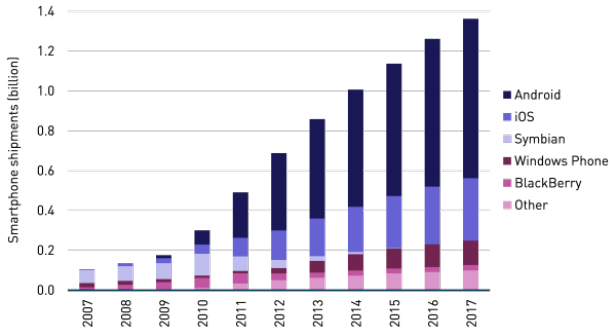


FIGURE – Smartphone shipments by operating system, worldwide, 2007-2017

- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement
- 4 Interface Graphique
- 5 Manipulez les Périphériques & Capteurs
- 6 Labo
- 7 Communiquez en Bluetooth & SMS
- 8 Publiez & Monétisez

Setup

- ▷ JAVA
- ▷ SDK Android
- ▷ IDE : Android Studio
- ▷ Emulateur

<https://developer.android.com/studio/index.html>

Android version



FIGURE – Different versions

Compilation

- ▷ .java → .class
- ▷ .class → .dex (Bytecode Dalvik)
- ▷ .apk == package
 - ◇ .dex
 - ◇ ressources (img ...)
 - ◇ AndroidManifest.xml

- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement**
- 4 Interface Graphique
- 5 Manipulez les Périphériques & Capteurs
- 6 Labo
- 7 Communiquez en Bluetooth & SMS
- 8 Publiez & Monétisez

Application type

- ▷ Premier plan (ex : jeu de poker)
- ▷ Arrière plan (ex : répondeur auto aux SMS)
- ▷ Intermittente (ex : lecteur multimedia)
- ▷ Widget (ex : météo du jour)

Activity

- ▶ Une activité == un écran graphique (== 1 use case UML)
- ▶ Une application est formée de n activités
- ▶ Etend `AppCompatActivity`
- ▶ Une activité lance une autre activité

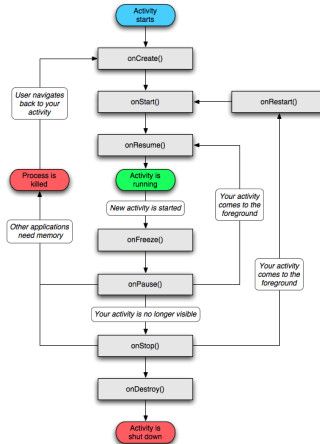


FIGURE – Activity life cycle Android

Callback

```
void onCreate(...)  
void onStart()  
void onRestart()  
void onResume()  
void onPause()  
void onStop()  
void onDestroy()
```

Bundle pour mémoriser l'état de l'activité (passage arrière plan)

Activity

```
package fr.enib.buche;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

public class Home extends AppCompatActivity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        /* Allocation des ressources ici */
    }

    @Override
    public void onResume() {
        /* Preparation des vues ici */
    }

    @Override protected void onDestroy() {
        super.onDestroy();
        /* Desallocation des ressources ici */
    }
}
```

La première activité est définie dans le manifest

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8">

<manifest package="fr.enib.buche">
    <uses-permission />
    <uses-sdk />
    <supports-screens />

    <application>
        <activity>
            <intent-filter>
                <action />
                <category />
                <data />
            </intent-filter>
        </activity>

        <service>
            <intent-filter> . . . </intent-filter>
        </service>

        <receiver>
            <intent-filter> . . . </intent-filter>
        </receiver>

        <provider>
            <grant-uri-permission />
        </provider>

    </application>
</manifest>
```

Context

Environnement de l'application

```
getResources  
getPackageName  
getSystemService  
  
startActivity, startService, sendBroadcast  
  
openFileInput, getSharedPreferences
```

Accès :

- ▶ Activity ou service : this (car héritage)
- ▶ BroadcastReceiver : argument de onReceive()
- ▶ ContentProvider : this.getContext()

Service

- ▷ == une activité sans GUI (ex : long calcul)
- ▷ exécution en tâche de fond
- ▷ `LocalService` dans le même processus que l'application
- ▷ `RemoteService` dans un processus indépendant
- ▷ Etend `android.app.Service` ou `IntentService`

Service

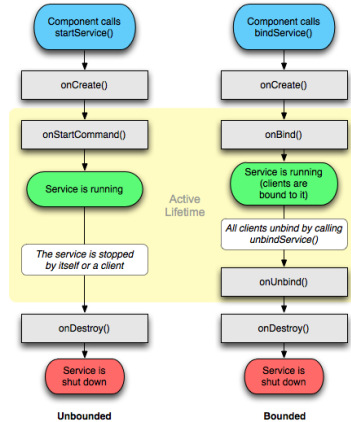


FIGURE – Service life cycle Android

Service

```
package fr.enib.cedric;

import android.app.Service;

public class AccountCleaner extends Service {

    @Override
    public void onCreate() {
        /* Allocation des ressources ici */
    }

    @Override int onStartCommand(Intent intent, int flags, int startId) {
        /* Votre code du service ici */
    }

    @Override protected void onDestroy() {
        super.onDestroy();
        /* D\'esallocation des ressources ici */
    }
}
```

Intent

- ▷ Active soit :
 - ◇ Une Activity
 - ◇ Un Service
 - ◇ Un BroadcastReceiver
- ▷ Message qui contient des informations : actions à réaliser, événement, Bundle ...
- ▷ Reception :
 - ◇ `<intent-filter>`
 - ◇ Extends `BroadcastReceiver`

Intent

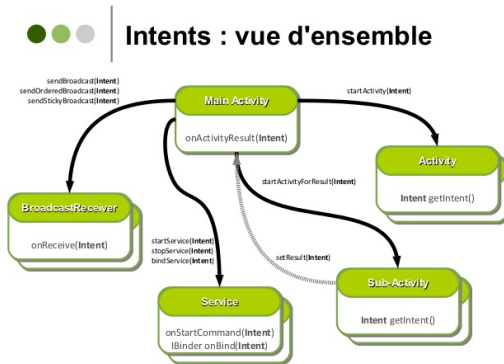


FIGURE – Intents overview

BroadcastReceiver

- ▷ Processus inactif
- ▷ Attend la réception d'un événement à la suite d'un `sendBroadcast(...)`
- ▷ Possibilité de lancer un activité
- ▷ Une seule méthode : `onReceive`
- ▷ Ne vit que le temps de cette méthode

BroadcastReceiver

```
package fr.enib.cedric;  
  
import android.content.BroadcastReceiver;  
  
public class Sleep extends BroadcastReceiver {  
  
    @Override  
    public void onReceive(Context arg0, Intent arg1) {  
        Intent i = new Intent(arg0, AccountClearn.class);  
        arg0.startService(i);  
    }  
}
```

- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement
- 4 Interface Graphique**
- 5 Manipulez les Périphériques & Capteurs
- 6 Labo
- 7 Communiquez en Bluetooth & SMS
- 8 Publiez & Monétisez

View

Interface : View ou ViewGroup (Layout)

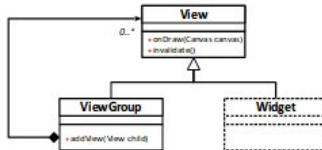


FIGURE – Android view

ViewGroup

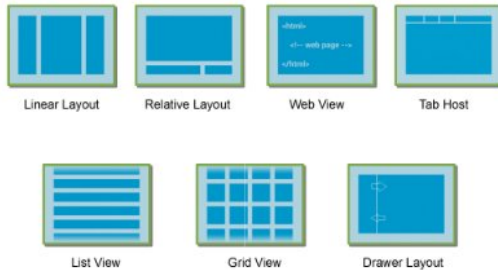


FIGURE – Android ViewGroup

Programme ViewGroup

```
public class Programme extends Activity {  
    public void onCreate(Bundle s) {  
        super.onCreate(s) ;  
        LinearLayout l = newLinearLayout(this) ;  
        TextView t = newTextView(this) ;  
        t.setText("ENIB" ) ;  
        EditText n = newEditText(this) ;  
        l.addView(t) ;  
        l.addView(n) ;  
        setContentView(l);  
    }  
}
```

Widget



FIGURE – Android ViewGroup

Déclaration XML

```
<?xml version="1.0" encoding="utf-8"?>  
  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    android:orientation="vertical"  
    android:layout_width="fill_parent"  
    android:layout_height="fill_parent">  
    <TextView android:id="@+id/textvname" android:  
      layout_width="fill_parent"  
      android:layout_height="wrap_content"  
      android:text="@string/name" />  
    <EditText android:id="@+id/edname"  
      android:layout_width="fill_parent"  
      android:layout_height="wrap_content"/>  
  </LinearLayout>
```

```
public class Programme extends Activity {  
  public void onCreate(Bundle s) {  
    super.onCreate(s) ;  
    setContentView(R.layout.main);  
  }  
}
```


R.java

- ▶ Chaque élément défini dans le répertoire `/res` impacte le fichier `R.java` (à ne pas toucher)
- ▶ Chemin d'accès aux ressources via `R.java`
- ▶ Objet java représentant les ressources
 - ◇ Instance de la classe `android.content.res.Resources`
 - ◇ Ressources du projet en cours : `context.getResources()`

Ressources

- ▷ menu/
 - ◇ Fichiers XML qui décrivent des menus de l'application
- ▷ raw/
 - ◇ Fichiers propriétaires (mp3, pdf, ...)
- ▷ values- [qualifier] /
 - ◇ Fichiers XML pour différentes sortes de ressources (textes, styles, dimensions, ...)
- ▷ xml/
 - ◇ Divers fichiers XML qui servent à la configuration de l'application (préférences, infos BDD, ...)
- ▷ anim/
 - ◇ Fichiers XML qui sont compilés en objets d'animation

Ressources

- ▷ `color/`
 - ◇ Fichiers XML qui décrivent des couleurs
- ▷ `drawable- [qualifier] /`
 - ◇ Fichiers bitmap (PNG, JPEG, or GIF), 9-Patch, ou fichiers XML qui décrivent des objets dessinables
- ▷ `layout- [qualifier] /`
 - ◇ Fichiers XML compilés en vues (écrans, fragments, ...)

Exploiter les ressources

- ▷ Utiliser des ressources depuis le code
 - ◇ La plupart des éléments de l'API sont prévus pour accepter des ressources Android en paramètre (`int ressource`)
 - ◇ `obj.setColor(R.color.rose_bonbon);`
 - ◇ `obj.setColor(android.R.color.darker_gray);`
- ▷ Référencer des ressources depuis d'autres ressources
 - ◇ `attribute="@[packageName]:resourcetype/ressourceIdent"`
 - ◇ `<EditText android:textColor="@color/rose_bonbon"/>`
 - ◇ `<EditText`
`android:textColor="@android:color/darker_gray"/>`

2D

- ▷ Une classe qui hérite de `SurfaceView`
- ▷ Sensible aux événements (`onTouch ...`)
- ▷ La surface de rendu est gérée par un `SurfaceHolder` (accès par `thread`)
- ▷ Contrôle du rendu par `SurfaceHolder.Callback`
 - ◇ `surfaceChanged()`
 - ◇ `surfaceCreated()`
 - ◇ `surfaceDestroyed()`

2D

```
public class MySurfaceView extends SurfaceView implements SurfaceHolder.Callback{
    public MySurfaceView(Context context){
        setWillNotDrawable(false);
        holder = getHolder();
        holder.addCallback(this);
    }

    @Override
    public void surfaceChanged(SurfaceHolder holder, int format, int width, int height){
        /* do something */
    }

    @Override
    public void surfaceCreated(SurfaceHolder holder){
        /* do something */
    }

    @Override
    public void surfaceDestroyed(SurfaceHolder holder){
        /* do something */
    }

}
```

2D

- ▷ Canvas : zone de dessin
- ▷ Dans un autre thread :
 - ◇ `SurfaceHolder.lockCanvas()`
 - ◇ `SurfaceHolder.unlockCanvasAndPost()`
 - ◇ Dans un `synchronized` / `finally`

2D

```
Canvas c = null;
try {
    c = holder.lockCanvas(null);
    synchronized(holder) {
        doDraw(c);
    }
} finally {
    if (c != null)
    {
        holder.unlockCanvasAndPost(c);
    }
}
```


- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement
- 4 Interface Graphique
- 5 Manipulez les Périphériques & Capteurs**
- 6 Labo
- 7 Communiquez en Bluetooth & SMS
- 8 Publiez & Monétisez

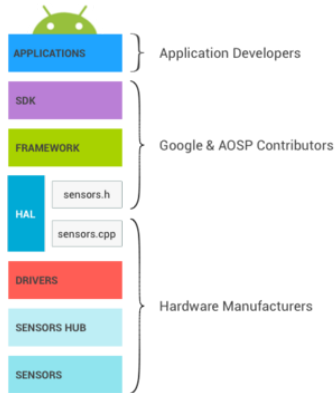


FIGURE – Architecture logicielle des capteurs.

Catégories

- ▷ capteurs de mouvement
- ▷ capteurs environnementaux
- ▷ capteurs de position

Les capteurs de mouvement

- ▷ accéléromètre (variations de vitesse) : trois dimensions x,y,z
- ▷ gyromètre (variations de rotations autour d'un axe) : trois dimensions x,y,z

Les capteurs environnementaux

- ▷ température
- ▷ pression atmosphérique
- ▷ éclairage
- ▷ humidité

Capteurs de position

- ▷ accéléromètre,
- ▷ le magnétomètre,
- ▷ le détecteur de proximité

Système de coordonnées

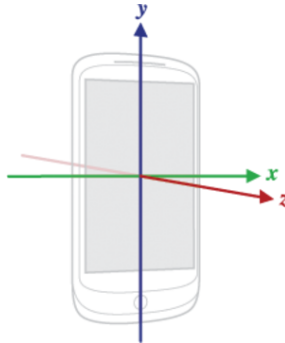


FIGURE – Système de coordonnées.

Système de coordonnées

- ▷ mode portrait pour les smartphones,
- ▷ en mode paysage pour les tablettes
- ▷ x : roll
- ▷ y : yaw
- ▷ z : pitch

Représentation dans le repère

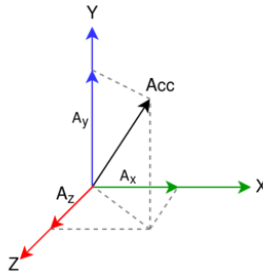


FIGURE – Représentation d'un capteur 3D.

$$||Acc|| = \sqrt{Ax^2 + Ay^2 + Az^2}$$

au repos : accélération verticale sur Z de $||Acc|| = +9.81m/s^2$, soit $1g$

Calibration des capteurs

$$Val = c * Acc + b + \epsilon$$

- ▷ avec Val la valeur mesurée finale avec erreur,
- ▷ c le gain,
- ▷ Acc la mesure réelle sans erreur,
- ▷ b le biais et
- ▷ ϵ le bruit blanc de moyenne zéro.

$$\begin{pmatrix} Val_x \\ Val_y \\ Val_z \end{pmatrix} = \begin{pmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & c_z \end{pmatrix} \begin{pmatrix} Acc_x \\ Acc_y \\ Acc_z \end{pmatrix} + \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix} + \begin{pmatrix} \epsilon_x \\ \epsilon_y \\ \epsilon_z \end{pmatrix}$$

manifest

- ▶ L'accès aux capteurs requiert parfois une permission comme le capteur *HEART_RATE*.
- ▶ Lorsqu'on utilise des capteurs dans une application, il est préférable de le déclarer dans le manifest. Cela permet de réaliser un filtre sur le "store" pour les systèmes qui ne possèdent pas un capteur.
- ▶ Ici un exemple avec l'accéléromètre et le gyromètre :

```
<uses-feature android:name="android.hardware.sensor.accelerometer" android:required="true"
/>

<uses-feature android:name="android.hardware.sensor.compass" android:required="false" />
```

APIs : Détection des capteurs présents

```
package fr.enib.buche;
import android.content.Context;
import android.hardware.Sensor;
import android.hardware.SensorManager;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.util.Log;
import java.util.List;

public class MainActivity extends AppCompatActivity {
    final String TAG="sensor";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        sensorDetection();
    }
}
```

APIs : Détection des capteurs présents

```
void sensorDetection(){  
  
    SensorManager mSensorManager;  
    mSensorManager = (SensorManager) getSystemService(Context.SENSOR_SERVICE);  
    List<Sensor> deviceSensors = mSensorManager.getSensorList(Sensor.TYPE_ALL);  
    if(deviceSensors!=null && !deviceSensors.isEmpty()) {  
        for (Sensor mySensor : deviceSensors) {  
            Log.v(TAG, "info:" + mySensor.toString());  
        }  
  
        if (mSensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER) != null){  
            Log.v(TAG, "info:Accelerometer found!");  
        }  
  
        else {  
            Log.v(TAG, "info:Accelerometer not found!");  
        }  
    }  
}
```

APIs : Détection des capteurs présents

```
info: {Sensor name="LGE_Accelerometer_Sensor", vendor="InvenSense", version=1, type=1, maxRange=39.226593, resolution=0.0011901855, power=0.5, minDelay=5000}
```

...

APIs : Acquisition d'un capteur

```
package fr.enib.cedric;
...
public class MainActivity extends AppCompatActivity implements SensorEventListener {
    final String TAG="sensor";
    SensorManager mSensorManager;
    private Sensor mAccelerometer;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        sensorDetection();
        if(mSensorManager==null) {
            mSensorManager = (SensorManager) getSystemService(Context.SENSOR_SERVICE);
        }
        mAccelerometer = mSensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
    }
}
```

APIs : Acquisition d'un capteur

```
@Override
protected void onResume() {
    super.onResume();
    if(mSensorManager!=null) {
        mSensorManager.registerListener(this, mAccelerometer, SensorManager.
            SENSOR_DELAY_NORMAL);
    }
}

@Override
protected void onPause() {
    super.onPause();
    if(mSensorManager!=null) {
        mSensorManager.unregisterListener(this);
    }
}
```


APIs : Acquisition d'un capteur

```
@Override
public final void onAccuracyChanged(Sensor sensor, int accuracy) {
    // Do something here if sensor accuracy changes.
}

@Override
public final void onSensorChanged(SensorEvent event) {
    // Many sensors return 3 values, one for each axis.
    float Ax = event.values[0];
    float Ay = event.values[1];
    float Az = event.values[2];
    // Do something with this sensor value.
    Log.v(TAG, "TimeAcc=" + event.timestamp + " Ax=" + Ax + " Ay=" + Ay + " Az=" + Az);
}

...
}
```

APIs : Acquisition d'un capteur

```
Time = 1488810980149390 Ax = 0.094680 Ay = -0.149749 Az = 9.482895  
Time = 1488810980215943 Ax = 0.100631 Ay = -0.140228 Az = 9.507889  
...
```

APIs : Acquisition multi-capteurs

```
package fr.enib.cedric;
...
public class MainActivity extends AppCompatActivity implements SensorEventListener {
    final String TAG="sensor";
    SensorManager mSensorManager;
    private Sensor mAccelerometer;
    private Sensor mGyroscope;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        ...
        mGyroscope = mSensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
    }

    @Override
    protected void onResume() {
        super.onResume();
        if(mSensorManager!=null) {
            mSensorManager.registerListener(this, mAccelerometer, SensorManager.
                SENSOR_DELAY_NORMAL);
            mSensorManager.registerListener(this, mGyroscope, SensorManager.SENSOR_DELAY_NORMAL);
        }
    }
}
```

APIs : Acquisition multi-capteurs

...

```
@Override
public final void onSensorChanged(SensorEvent event) {
    if(event.sensor.getType()== Sensor.TYPE_ACCELEROMETER) {
        // TYPE_ACCELEROMETER -> 3 values, one for each axis
        float Ax = event.values[0];
        float Ay = event.values[1];
        float Az = event.values[2];
        // Do something with this sensor value.
        Log.v(TAG, "TimeAcc_=" + event.timestamp + "_Ax=" + Ax + "_" + "Ay=" + Ay + "_"
            + "Az=" + Az);
    }

    if(event.sensor.getType()== Sensor.TYPE_GYROSCOPE)
    {
        // TYPE_GYROSCOPE -> 3 values, one for each axis
        float Gx = event.values[0];
        float Gy = event.values[1];
        float Gz = event.values[2];
        // Do something with this sensor value.
        Log.v(TAG, "TimeGyro_=" + event.timestamp + "_Gx=" + Gx + "_" + "Gy=" + Gy + "_"
            + "Gz=" + Gz);
    }

}

}

...

}
```

APIs : Acquisition multi-capteurs

manifest

```
<uses-feature android:name="android.hardware.sensor.gyroscope" android:required="true" />
```

APIs : Acquisition multi-capteurs

```
TimeGyro = 1488811983068660436 Gx = -0.0072784424 Gy = 0.0044403076 Gz = -9.613037E-4  
TimeAcc = 1488811983159114537 Ax = 0.07563782 Ay = -0.16998291 Az = 9.528122  
...
```

- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement
- 4 Interface Graphique
- 5 Manipulez les Périphériques & Capteurs
- 6 Labo**
- 7 Communiquez en Bluetooth & SMS
- 8 Publiez & Monétisez

Sujet 1 : Dessin 2D

- ▶ Créer un layout soit dans le code soit avec l'interface de visual studio (au choix)
- ▶ Dessiner des formes géométriques (exemples : cercles, carrés ..)
- ▶ Comportement des formes dans un thread (déplacement autonome des formes)
- ▶ Gestion des rebonds (murs)
- ▶ Interaction selon `TouchEvent` (déplacement contrôlé par l'utilisateur)

Sujet 2 : Dessin 2D + vibreur + capteur

- ▷ Faire vibrer (Vibrator service `VIBRATOR_SERVICE`) quand la forme touche les murs, les bords ... (rebonds)
- ▷ Faire bouger la forme sur la tablette selon une orientation

Sujet 3 : Camera

- ▷ Afficher la preview de la camera (`android.hardware.Camera`) dans un autre layout
- ▷ Récupérer le flux, puis appliquer un traitement (seuil, niveau de gris, contours)
- ▷ Afficher sur la vidéo un objet2D
- ▷ Selon l'orientation de la tablette, modifier la position de l'objet2D

- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement
- 4 Interface Graphique
- 5 Manipulez les Périphériques & Capteurs
- 6 Labo
- 7 Communiquez en Bluetooth & SMS**
- 8 Publiez & Monétisez

Intégration du Bluetooth

Bluetooth == Communication courte distance
Permissions (Manifest) :

```
...  
<uses-permission android:name="android.permission.BLUETOOTH" />  
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />  
...
```

vérifier la présence du package Bluetooth

```
mBluetoothAdapter = BluetoothAdapter.getDefaultAdapter();  
if(mBluetoothAdapter==null) {  
    Toast.makeText(this, "Device does not support Bluetooth", Toast.LENGTH_LONG).show();  
}
```

Intégration du Bluetooth

mettre en marche le système Bluetooth

```
private static final int ENABLE_BLUETOOTH = 1;
private void initBluetooth() {
    if (!mBluetoothAdapter.isEnabled()) {
        Intent intent = new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
        startActivityForResult(intent, ENABLE_BLUETOOTH);
    }
}
```

Intégration du Bluetooth

```
...  
  
protected void onActivityResult(int requestCode, int resultCode, Intent data) {  
    if (requestCode == ENABLE_BLUETOOTH)  
        if (resultCode == RESULT_OK) {  
            Log.v(TAG, "BT_␣" + ENABLE_BLUETOOTH);  
        }  
  
    if (requestCode == DISCOVERY_REQUEST) {  
        if (resultCode == RESULT_CANCELED) {  
            Log.d(TAG, "Discovery_␣cancelled_␣by_␣user");  
        } else {  
            Log.v(TAG, "Discovery_␣allowed");  
        }  
    }  
}
```

Nous pourrions capturer si l'utilisateur a appuyé sur Refuser ou Accepter.

Intégration du Bluetooth

Autoriser à répondre aux autres équipements présents

```
private void makeDiscoverable() {  
    Intent discoverableIntent = new Intent(BluetoothAdapter.ACTION_REQUEST_DISCOVERABLE);  
    //discoverable for 5 minutes (~300 seconds)  
    discoverableIntent.putExtra(BluetoothAdapter.EXTRA_DISCOVERABLE_DURATION, 300);  
    startActivityForResult(discoverableIntent, DISCOVERY_REQUEST);  
    Log.i("Log", "Discoverable_");  
}
```

Intégration du Bluetooth

- ▶ canal de communication radio Bluetooth est découpé en 79 canaux
- ▶ chaque équipement utilise l'un de ces canaux.
- ▶ 32 canaux sont utilisés pour réaliser la découverte des équipements voisins et établir une communication.
- ▶ Un équipement envoie des messages de découverte sur l'un de ces canaux et un autre équipement écoute périodiquement et répond à ces messages.
- ▶ La méthode `startDiscovery()` du `BluetoothAdapter` permet de lancer le scan sur tout le spectre radio Bluetooth.

Intégration du Bluetooth

```
...  
  
mBluetoothAdapter = BluetoothAdapter.getDefaultAdapter();  
  
if (mBluetoothAdapter == null) {  
    Toast.makeText(this, "Device does not support Bluetooth", Toast.LENGTH_LONG).show();  
}  
  
//start discovery  
mBluetoothAdapter.startDiscovery();  
  
...
```

Intégration du Bluetooth

Pour récupérer les équipements découverts par le scan, il faut implémenter un `broadcastReceiver` qui reçoit les Intent du système. Le `broadcastReceiver` reçoit également les événements sur l'état des connexions BT :

- ▶ `ACTION_DISCOVERY_STARTED` : début de la recherche des équipements ;
- ▶ `ACTION_DISCOVERY_FINISHED` : fin de la recherche ;
- ▶ `ACTION_BOND_STATE_CHANGED` : changement de la connexion ;
- ▶ `BOND_BONDED` : connexion établie ;
- ▶ `BOND_NONE` : connexion non établie.

Intégration du Bluetooth

```
...  
// Register for broadcasts when a device is discovered.  
IntentFilter filter = new IntentFilter();  
...  
filter.addAction(BluetoothAdapter.ACTION_STATE_CHANGED);  
filter.addAction(BluetoothDevice.ACTION_FOUND);  
filter.addAction(BluetoothAdapter.ACTION_DISCOVERY_STARTED);  
filter.addAction(BluetoothAdapter.ACTION_DISCOVERY_FINISHED);  
filter.addAction(BluetoothDevice.ACTION_BOND_STATE_CHANGED);  
registerReceiver(mReceiver, filter);  
  
private final BroadcastReceiver mReceiver = new BroadcastReceiver() {  
    public void onReceive(Context context, Intent intent) {  
        String action = intent.getAction();  
        if (BluetoothAdapter.ACTION_DISCOVERY_STARTED.equals(action)) {  
            msgText.setText("Discovery started");  
            Log.v(TAG, "ACTION_DISCOVERY_STARTED");  
            //discovery starts, we can show progress dialog or perform other tasks  
        } else if (BluetoothAdapter.ACTION_DISCOVERY_FINISHED.equals(action)) {  
            //discovery finishes, dismiss progress dialog  
            msgText.setText("Discovery finished");  
            Log.v(TAG, "ACTION_DISCOVERY_FINISHED");  
        }  
    }  
}
```

Intégration du Bluetooth

```
...
} else if (BluetoothDevice.ACTION_FOUND.equals(action)) {
    //bluetooth device found
    BluetoothDevice device = (BluetoothDevice) intent.getParcelableExtra(BluetoothDevice.
        EXTRA_DEVICE);
    if(!deviceListName.isEmpty() && firstElement) {
        firstElement = false;
        deviceListName.clear();
    }

    deviceBT.add(device);
    deviceListName.add(device.getName());
    arrayAdapter.notifyDataSetChanged();
    Log.v(TAG, "Found device " + device.getName());
} else if (BluetoothDevice.ACTION_BOND_STATE_CHANGED.equals(action)) {
    final int state = intent.getIntExtra(BluetoothDevice.EXTRA_BOND_STATE,
        BluetoothDevice.ERROR);
    final int prevState = intent.getIntExtra(BluetoothDevice.EXTRA_PREVIOUS_BOND_STATE,
        BluetoothDevice.ERROR);
    if (state == BluetoothDevice.BOND_BONDED) {
        Log.v(TAG, "Paired");
    } else if (state == BluetoothDevice.BOND_NONE && prevState == BluetoothDevice.
        BOND_BONDED) {
        Log.v(TAG, "Unpaired");
    }
}
}
};
...
```

Intégration du Bluetooth

Des équipements sont associés s'ils ont réalisé la phase de découverte mutuelle et ont effectué une demande d'association, alors ils sont enregistrés par le système comme équipements associés.

```
private Set<BluetoothDevice> getPairedDevices(BluetoothAdapter mBluetoothAdapter)
{
    Set<android.bluetooth.BluetoothDevice> pairedDevices = mBluetoothAdapter.getBondedDevices();
    if (pairedDevices.size() > 0) {
        // There are paired devices. Get the name and address of each paired device.
        for (android.bluetooth.BluetoothDevice device : pairedDevices) {
            Log.v(TAG, "paired_" + device.getName());
            Log.v(TAG, "paired_" + device.getAddress());
            deviceBT.add(device);
            deviceListName.add(device.getName() + " paired");
            arrayAdapter.notifyDataSetChanged();
        }
    }
    return pairedDevices;
}
```

Intégration du Bluetooth

Association BT :

```
private void pairDevice(BluetoothDevice device) {  
    try {  
        Method method = device.getClass().getMethod("createBond", (Class[]) null);  
        method.invoke(device, (Object[]) null);  
  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

Intégration du Bluetooth

Suppression de l'association BT

```
private void unpairDevice(BluetoothDevice device) {  
    try {  
        Method method = device.getClass().getMethod("removeBond", (Class[]) null);  
        method.invoke(device, (Object[]) null);  
  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

Structure client/serveur

chaque smartphone peut être client/serveur

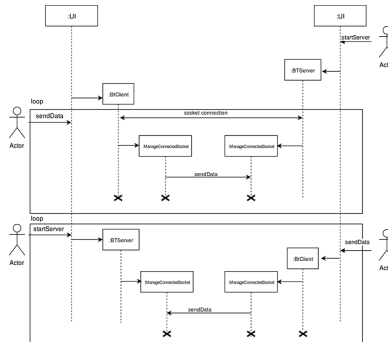


FIGURE –

Bluetooth - Structure serveur

```
private class BtServer extends Thread {
    BluetoothServerSocket mmServerSocket = null;
    public BtServer(BluetoothAdapter mBtAdapter, UUID mUUID) {
        UUID MY_UUID = mUUID;
        BluetoothAdapter mBluetoothAdapter = mBtAdapter;
        try {
            Log.v(TAG, "mBluetoothAdapter...");
            mmServerSocket = mBluetoothAdapter.listenUsingRfcommWithServiceRecord("server",
                MY_UUID);
            Log.v(TAG, "mBluetoothAdapter...ok");
        } catch (IOException e) {
            Log.e(TAG, "Socket's listen() method failed", e);
        }
    }
}
```

Bluetooth - Structure serveur

```
public void run() {  
    BluetoothSocket socket = null;  
    // Keep listening until exception occurs or a socket is returned.  
    while (true) {  
        try {  
            Log.v(TAG, "wait_socket");  
            socket = mmServerSocket.accept();  
            Log.v(TAG, "SocketAccepted");  
            Log.v(TAG, "Manage_socket");  
            mManageConnectedSocket = new ManageConnectedSocket(socket);  
            mManageConnectedSocket.start();  
  
        } catch (IOException e) {  
            // msgText.setText("BT socket closed or timeout");  
            Log.e(TAG, "Socket's accept() method failed", e);  
            break;  
        }  
    }  
}  
// Closes the connect socket and causes the thread to finish.  
public void cancel() {  
    try {  
        mmServerSocket.close();  
    } catch (IOException e) {  
        Log.e(TAG, "Could not close the connect socket", e);  
    }  
}  
}
```

Bluetooth - Structure client

```
private class BtClient extends Thread {
    private BluetoothSocket mmSocket;
    private final BluetoothDevice mmDevice;
    BluetoothAdapter mBluetoothAdapter;
    UUID mUUID;
    public BtClient(BluetoothDevice device, BluetoothAdapter myBluetoothAdapter, UUID MY_UUID) {
        mmDevice = device;
        mUUID = MY_UUID;
        mBluetoothAdapter = myBluetoothAdapter;
        try {
            // Get a BluetoothSocket to connect with the given BluetoothDevice.
            // MY_UUID is the app's UUID string, also used in the server code.
            mmSocket = device.createRfcommSocketToServiceRecord(mUUID);
            Log.v(TAG, "Client_getBluetoothRemoteServer_" + mmSocket.getRemoteDevice().getName());
        } catch (IOException e) {
            Log.e(TAG, "Socket's_create()_method_failed", e);
        }
    }
}
```

Bluetooth - Structure client

```
public void run() {  
    // Cancel discovery because it otherwise slows down the connection.  
    mBluetoothAdapter.cancelDiscovery();  
    try {  
        // Connect to the remote device through the socket. This call blocks  
        // until it succeeds or throws an exception.  
        Log.v(TAG, "Client_try_to_connected_to_server");  
        mmSocket.connect();  
        Log.v(TAG, "Client_socket_connected");  
        // The connection attempt succeeded. Perform work associated with  
        // the connection in a separate thread.  
        mySocketManager = new ManageConnectedSocket(mmSocket);  
        mySocketManager.start();  
    } catch (IOException connectException) {  
        // Unable to connect; close the socket and return.  
        try {  
            mmSocket.close();  
        }  
        catch (IOException closeException) {  
            Log.e(TAG, "Could_not_close_the_client_socket", closeException);  
        }  
        return;  
    }  
}  
...
```

Bluetooth - Structure client

```
...  
  
// Closes the client socket and causes the thread to finish.  
public void cancel() {  
    try {  
        mmSocket.close();  
  
    } catch (IOException e) {  
        Log.e(TAG, "Could not close the client socket", e);  
    }  
}  
}
```

Transfert des données

écrit dans la socket Bluetooth avec la méthode `write(String msg)` et
lit avec `getInputStream()`

Send SMS

```
private void sendSMS()
{
    final int PERMISSION_REQUEST_CODE = 1;
    if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.M) {
        if (checkSelfPermission(android.Manifest.permission.SEND_SMS) == PackageManager.
            PERMISSION_DENIED) {
            Log.d("permission", "permission_déniée pour SEND_SMS - requestant");
            String[] permissions = {android.Manifest.permission.SEND_SMS};
            requestPermissions(permissions, PERMISSION_REQUEST_CODE);
        }
    }

    String msg = "Hello";
    Intent smsIntent = new Intent(Intent.ACTION_SENDTO);
    smsIntent.addCategory(Intent.CATEGORY_DEFAULT);
    smsIntent.setType("vnd.android-dir/mms-sms");
    smsIntent.putExtra("sms_body", msg);
    smsIntent.setData(Uri.parse("sms:"));
    startActivity(smsIntent);
}
```

- 1 Introduction
- 2 Environnements de développement
- 3 Principes de développement
- 4 Interface Graphique
- 5 Manipulez les Périphériques & Capteurs
- 6 Labo
- 7 Communiquez en Bluetooth & SMS
- 8 Publiez & Monétisez

Préparez

Avant de diffuser votre application, il va encore falloir

- ▷ la tester,
- ▷ générer un certificat,
- ▷ signer l'application,
- ▷ l'optimiser.

Identification

Identification : fichier build.gradle

```
android {  
    signingConfigs {  
        release {  
            keyAlias 'keyRelease'  
            storeFile file('/home/android/.android/releasekey.jks')  
        }  
    }  
    compileSdkVersion 23  
    buildToolsVersion '25.0.0'  
    defaultConfig {  
        applicationId "cedric.app.hs.store"  
        minSdkVersion 16  
        targetSdkVersion 23  
        versionCode 1  
        versionName "1.0"  
        signingConfig signingConfigs.release  
    }  
}
```

Utilisé par Google Play Console lors du déploiement

Google Play Console

- ▶ `applicationId` : permet de différencier l'application des autres sur le store de Google. Cet identifiant doit toujours être le même et doit être signé avec le même certificat, si l'application ID ou le certificat sont différents alors l'application est considérée comme une nouvelle application.
- ▶ `versionCode` : représente le numéro de version interne (entier entre 1 et 2 100 000 000). Information utilisée comme référence uniquement pour le développement de l'application.
- ▶ `versionName` : c'est la version officielle qui est présentée aux utilisateurs. En principe, la syntaxe est la suivante `< major > . < minor >`. Elle peut être synchronisée avec `versionCode`, mais ce n'est pas une obligation.

Tests unitaires locaux

Dans module-name/src/test/java
Configurer fichier build.gradle

```
dependencies {  
    ...  
    // Test unitaire avec JUNIT  
    testCompile 'junit:junit:4.12'  
}
```

Dans votre code, ouvrir le fichier, appuyez sur Control + Shift + t, choisir la méthode à tester puis OK. Un test vide est alors créé dans le répertoire de test

```
public class Toto {  
    @ Test  
    ....  
}
```

Tests instrumentés

Dans module-name/src/androidTest/java
Configurer fichier build.gradle

```
defaultConfig {  
    ...  
    testIntrumentationRunner "android.support.test.runner.AndroidJUnitRunner"  
}
```

AndroidJUnitRunner fournit par AndroidStudio tests JUnit-3, JUnit-4,
Espresso et UI Automator

Type de build debug/release

- ▶ Dans la phase de développement, le mode debug permet de réaliser des points d'arrêt avec le debugger, de visualiser les exceptions.
- ▶ Par défaut, Android Studio crée un projet avec les modes release et debug.

Certificat

- ▶ Le certificat est composé d'une clé publique qui est diffusée et d'une clé privée qui est secrète et protégée par un mot de passe.
- ▶ En mode release, le certificat doit être généré avec de vrais identifiants (par défaut l'émetteur est Android avec un mot de passe android).
- ▶ `$ keytool -genkey -v -keystore release.keystore -alias releasekey -keyalg RSA -keysize 2048 -validity 10000`
- ▶ androidStudio : Build > Generate Signed APK > Create New, puis saisir le formulaire de la fenêtre New Key Store.
- ▶ Le fichier de stockage généré contient une clé publique/privée.

Signer votre application

- ▷ Manuel : Build > Generate signed apk.
- ▷ Auto : File > Project Structure (onglet Signing)
- ▷ Pour créer un type de signature, cliquez sur + et saisissez les données du formulaire. Ensuite, dans l'onglet Build Types, il faut associer cette signature à un type de build.
Le fichier build.gradle est impacté. Une nouvelle section signingConfigs apparaît avec le détail de la configuration :

```
android {  
    signingConfigs {  
        release {  
            keyAlias 'relaseKey'  
            storeFile file('~/android/keystore.jks')  
            keyPassword 'myPass'  
            storePassword ''myPass''  
        }  
    }  
}
```


Supprimer les logs et android :debuggable

conditionner l'utilisation du logcat :

```
...  
  
if (BuildConfig.DEBUG) {  
    Log.v(TAG, intent.getAction());  
}  
  
...
```

Supprimer les logs et android :debuggable

- ▶ Le mode de build est choisi via le menu Build > Select Build Variant, sélectionner release ou debug.
- ▶ Dans app/build/generated/source/debug et app/build/generated/source/release, sous le nom de package, se trouve le fichier BuildConfig.java qui contient des variables. Ces deux fichiers sont utilisés selon le mode de build :

```
public final class BuildConfig{  
    public static final boolean DEBUG = false;  
    public static final String APPLICATION_ID ="linuxmag.app.hs.store.release";  
    public static final String BUILD_TYPE ="release";  
    public static final String FLAVOR ="";  
    public static final int VERSION_CODE =3;  
    public static final String VERSION_NAME ="1.2";  
}
```

Dans le fichier manifest embarqué dans l'archive APK, la directive android:debuggable doit être enlevée,

Choisir l'icône

- ▷ Choix est stratégique, car une fois déployé, il sera difficile de la changer, car elle deviendra vite un repère visuel.
- ▷ Google vous propose un template d'icône au format Photoshop qui vous permettra d'affiner votre choix.
- ▷ L'icône doit être générée dans différents formats correspondant à la densité d'affichage des différents équipements : ldpi (low), mdpi (medium), hdpi (high), xhdpi (extra-high), xxhdpi (extra-extra-high), et xxxhdpi (extra-extra-extra-high).

Diffusion de votre application

▷ Diffusion directe :

- ◇ la configuration des machines cibles doit être modifiée pour accepter les applications hors Google Play.
- ◇ Il faut autoriser les "sources inconnues" via le menu Paramètres > Sécurité > Unknown sources

Diffusion de votre application

▷ Diffusion sur un store :

- ◇ Google Play Console est le site de référence pour la diffusion de vos applications. Pour y accéder, vous devez au préalable créer un compte Google à cette adresse <https://accounts.google.com/SignUp>.
- ◇ Avec votre compte Google, créez ensuite un compte développeur à cette adresse <https://play.google.com/apps/publish>, c'est à partir de ce compte que vous pourrez démarrer le processus de diffusion. La création du compte développeur se déroule en quatre étapes et dure environ une dizaine de minutes :
 - ① Connexion avec votre compte Google,
 - ② Acceptez le contrat proposé par Google,
 - ③ Payez les frais d'inscription de 25\$,
 - ④ Renseignez votre compte.

Publiez votre première application

- ▶ Cliquez tout d'abord sur Publier une application Android sur Google Play. Précisez la langue par défaut puis, saisissez le nom de l'application (au maximum de 30 caractères), ensuite cliquez sur Créer. Ces paramètres seront modifiables par la suite. Chaque étape est ensuite validée par un « voyant vert » dans le menu de gauche.
- ▶ Dans la rubrique « Classification » de la fiche, un questionnaire est proposé afin de définir précisément les contenus. Avant de remplir ce formulaire, il faut envoyer l'archive APK via le menu Version de l'application. Vous pouvez décider si votre première publication est une version de production, bêta, ou alpha
- ▶ La fiche est maintenant validée. Allons maintenant à la rubrique « Classification » via le menu de gauche Classification du contenu. Le formulaire de classification est conséquent, tous les items de saisie sont obligatoires.

Modèles de tarification (1/2)

choix du type de diffusion gratuit/payant.

- ▶ menu Paramètres > Modèles de Tarification (prix, taxes)
- ▶ menu Tarifs et disponibilité
- ▶ Le compte marchand est nécessaire dès lors que vos applications sont payantes, Google utilisera les informations de ce compte pour vous payer les applications téléchargées à partir de Google Play.
- ▶ Pour configurer ce compte : allez dans le menu Paramètres > Détails du compte. Dans Compte marchand, cliquez sur Configurer votre compte marchand.

Modèles de tarification (2/2)

- ▶ Lorsque les étapes sont validées, vous pouvez demander la publication de votre application, car jusqu'à présent le contenu a été sauvegardé en mode "brouillon".
- ▶ À partir de la page de garde, sélectionnez votre application puis Gestion de la publication > Version de l'application, cliquez sur Lancer le déploiement ;

Programmation de systèmes mobiles — ANDROID — *CAI*

Cédric Buche

ENIB

30 mars 2020